LONG-TERM MEMORY ENGINE FOR AI AGENTS

A PROJECT REPORT

Submitted by

M. KRITHIKRAJ	**2023115004**
ALSTON S	**2023115006**
ARYAN I	**2023115021**
HARI SRINIVASAN	**2023115022**

submitted to the Faculty of

COMPUTER SCIENCE AND INFORMATION ENGINEERING

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY



DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY

COLLEGE OF ENGINEERING, GUINDY

ANNA UNIVERSITY

CHENNAI – 600 025

ANNA UNIVERSITY

CHENNAI – 600 025

BONA FIDE CERTIFICATE

Certified that this project report titled **LONG-TERM MEMORY ENGINE FOR AI AGENTS** is the bona fide work of **M. KRITHIKRAJ (2023115004), ALSTON S (2023115006), ARYAN I (2023115021)** and **HARI SRINIVASAN (2023115022)** who carried out the project work under my supervision. Certified further that to the best of my knowledge and belief, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or an award was conferred on an earlier occasion on this or any other candidate.

PLACE: Chennai

DATE:

DR. BAMA SRINIVASAN
PROJECT GUIDE
Department of Information Science and Technology
College of Engineering, Guindy
Anna University, Chennai – 600 025

COUNTERSIGNED

HEAD OF THE DEPARTMENT
Department of Information Science and Technology
College of Engineering, Guindy
Anna University, Chennai – 600 025

ABSTRACT

AI agents that don't have persistent memory forget instructions and project decisions once a session ends, and when an instruction changes they often keep using the old one. For this project we built a long-term memory engine in Rust for such agents. What it remembers carries over between sessions, can be corrected later, and can be traced back to the original messages.

The engine has five modules: source acquisition and processing, knowledge extraction and consolidation, temporal knowledge and graph management, hybrid retrieval and grounded reasoning, and model training and evaluation. Original messages are stored unchanged as source events. Assertions (typed statements about the user or project) are extracted along with the exact source quote they came from and stored as versions, so a correction replaces an old value without deleting it. Retrieval combines keyword, vector, time and graph search using Reciprocal Rank Fusion and returns each result with its source quotes. The extraction and answering is done by a small local Qwen3-4B model that we fine-tuned with QLoRA.

We evaluated the engine on LoCoMo, a public benchmark of ten long conversations (1,540 questions), with the same reader model under every condition and two independent model judges. With the engine the reader answered 83.1% of the questions correctly, against 80.0% with plain retrieval over the raw text, 43.0% when keeping only the most recent 8,192 tokens, and 0.3% with no memory. Giving the reader the whole conversation reached 89.0%, but needed more than three times as many tokens. The same picture held on the eight conversations never used for development, and every one of the 99,971 source quotes returned by the engine matched the conversation exactly. A deterministic property suite confirmed that corrections keep their history and

that the engine abstains without evidence. It also exposed a date-handling fault that was fixed after the test.

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, Dr. Bama Srinivasan, Department of Information Science and Technology, College of Engineering, Guindy, Anna University, for her guidance, regular reviews and constructive suggestions throughout this project. We also thank the Department of Information Science and Technology for the facilities and support provided.

We acknowledge the open-source and research community whose published work and software made this project possible, including the authors of the systems and papers cited in this report.

M. KRITHIKRAJ
ALSTON S
ARYAN I
HARI SRINIVASAN

TABLE OF CONTENTS

ABSTRACT iii
LIST OF TABLES viii
LIST OF FIGURES ix
LIST OF ABBREVIATIONS x
GLOSSARY xi

1 INTRODUCTION **1**
1.1 BACKGROUND 1
1.2 PROBLEM STATEMENT 2
1.3 OBJECTIVES 2
1.4 CHALLENGES FACED 3
1.5 ORGANISATION OF THE REPORT 3

2 LITERATURE SURVEY **5**
2.1 LONG CONTEXT AND EXTERNAL MEMORY 5
2.2 MEMORY SYSTEMS 5
2.3 METHODS USED IN THIS PROJECT 7
2.4 COMPARISON AND GAP 7

3 SYSTEM DESIGN **8**
3.1 DESIGN REQUIREMENTS 8
3.2 OVERALL ARCHITECTURE 8
3.3 M1 SOURCE ACQUISITION AND PROCESSING 9
3.4 M2 KNOWLEDGE EXTRACTION AND CONSOLIDATION 11
3.5 M3 TEMPORAL KNOWLEDGE AND GRAPH MANAGEMENT 12
3.6 M4 HYBRID RETRIEVAL AND GROUNDED REASONING 14
3.7 M5 MODEL TRAINING AND EVALUATION 14

4 IMPLEMENTATION **16**
4.1 IMPLEMENTATION ENVIRONMENT 16
4.2 ALGORITHMS 16
4.2.1 Priority-Based Context Selection 17
4.2.2 Entity Resolution and Memory Versioning 17
4.2.3 Memory Consolidation and Invalidation 18

4.2.4 Hybrid Retrieval and Graph Traversal 19
4.2.5 Reciprocal Rank Fusion and Context Packing 19
4.2.6 Source-Grounded Answer Generation 21
4.3 INTEGRATION WITH THE PI AGENT 21

5 RESULTS, PERFORMANCE ANALYSIS AND COMPARISON 22
5.1 DATASETS 22
5.2 FINE-TUNING AND MODEL SELECTION 24
5.3 LIVE DEMONSTRATION 25
5.4 EXPERIMENTAL SETUP 26
5.4.1 Conditions 27
5.4.2 Reader, judges and metrics 27
5.4.3 Protocol 28
5.4.4 Reproducibility 28
5.5 DEVELOPMENT ITERATIONS 29
5.6 RESULTS 30
5.6.1 Held-out conversations 33
5.6.2 Question categories 34
5.6.3 Ablations 35
5.6.4 Run-to-run variation and judge agreement 35
5.7 PERFORMANCE ANALYSIS 36
5.7.1 Retrieval and provenance 36
5.7.2 Tokens and latency 37
5.7.3 Cost of building memory 37
5.8 PROPERTY SUITE 38
5.9 COMPARISON WITH EXISTING SYSTEMS 40
5.10 THREATS TO VALIDITY 42
5.11 CONCLUSION AND FUTURE WORK 43

REFERENCES 45

LIST OF TABLES

2.1 Comparison of existing memory systems 7

5.1 Benchmark conditions 27
5.2 Changes made because of development-run findings 30
5.3 Success criteria fixed before the test run (Jev) 31
5.4 LoCoMo results by condition (all ten conversations, categories 1–4) 32
5.5 Accuracy of the memory engine minus each other condition, same questions (Jev) 33
5.6 Accuracy of the memory engine minus each other condition, same questions (GPT-4o-mini) 33
5.7 All ten conversations against the eight never used for diagnosis 34
5.8 Accuracy by LoCoMo question category (Jev, %) 35
5.9 Run-to-run variation: the reader asked three times on the 300-question sample (Jev) 36
5.10 Agreement of the two judges 36
5.11 Retrieval against LoCoMo's evidence labels (%) 37
5.12 Cost of building memory for LoCoMo on one RTX 4070 laptop GPU (8 GB) 38
5.13 Property suite: checks on the engine's stored state, no judge 40
5.14 Published LoCoMo results [5] beside this work (GPT-4o-mini judge) 41

LIST OF FIGURES

3.1 Overall system architecture 9
3.2 Module 1: Source Acquisition and Processing 10
3.3 Module 2: Knowledge Extraction and Consolidation 11
3.4 Module 3: Temporal Knowledge and Graph Management 13
3.5 Module 4: Hybrid Retrieval and Grounded Reasoning 14
3.6 Module 5: Model Training and Evaluation 15

5.1 General-purpose training rows across the 16 domains 23
5.2 Demo: recording details, then recalling them in a new session 25
5.3 Demo: recalled answer and the corrected assertion in the graph 26

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CPU	Central Processing Unit
DMR	Deep Memory Retrieval
GGUF	GPT-Generated Unified Format
HTTP	Hypertext Transfer Protocol
ID	Identifier
JSON	JavaScript Object Notation
LLM	Large Language Model
LoRA	Low-Rank Adaptation
MVCC	Multi-Version Concurrency Control
NELL	Never-Ending Language Learner
NF4	4-bit NormalFloat
p95	95th percentile
PROV	W3C Provenance data model
QLoRA	Quantised Low-Rank Adaptation
RRF	Reciprocal Rank Fusion
SHA-256	Secure Hash Algorithm, 256-bit
SLM	Small Language Model
SQL	Structured Query Language
W3C	World Wide Web Consortium

GLOSSARY

Some everyday computing words are used with a narrower meaning in this report. The meanings below are the ones we use throughout.

Memory	Knowledge about a user or project that the agent keeps across sessions (assertions, inferences and the source events they came from). It does *not* mean RAM, cache, disk or any other computer memory.
Long-term memory	Memory that survives after a session ends, as opposed to what is only in the current prompt.
Agent	An AI program (here, the Pi coding agent) that talks to the user and carries out tasks using a language model and tools.
Harness	The program around the model that runs the agent: it builds prompts, calls tools and in our case talks to the memory service.
Session	One continuous conversation between the user and the agent. A new session starts with an empty prompt.
Context / context window	The text the model can read at one time (its prompt). Its size is limited and measured in tokens.
Token	A small piece of text (a word or part of a word) that the model reads. Budgets and limits are counted in tokens.
Namespace	A label such as `user:maya` that keeps one user's or project's memory separate from another's.
Source event	A single message or tool result (with role and timestamp) exactly as it was sent, stored before anything is extracted from it. "Event" is short for source event.

Episode	One batch of source events saved together by a single retain request.
Source chunk	A piece of a source event's text that is stored and embedded for search.
Source quote	The exact passage of a source event that an assertion cites. It must appear word for word in the original text.
Spool	A local folder of files on the client where batches wait if the memory service can't be reached.
Job / worker	A job is a stored task (e.g. "extract assertions from this episode"); a worker is a background process that picks up and runs jobs.
Lease	A time-limited lock a worker takes on a namespace so that two workers don't process it at the same time.
Claim	A candidate assertion proposed by the model before the system has checked and stored it.
Assertion	A typed, temporal statement about an entity that the model extracted from source events, e.g. "Maya lives in Bengaluru", stored with its source quote and validity interval. It can be wrong, contested, superseded or retracted.
Attribute	The assertion kind for a stable property of an entity, such as a home city or an employer (stored as kind `attribute`).
Inference	An assertion (kind `inference`) derived from at least two supporting assertions, e.g. "Maya works remotely as a software engineer at NovaTech".
Entity	A person, project, place or thing that assertions are about.
Consolidation	The step that combines related assertions into an inference.

Invalidation	Marking an inference stale when one of the assertions it depends on changes.
Supersede	To replace an assertion's value with a newer one while keeping the old assertion as history.
Validity interval	The time range during which an assertion was true, e.g. Chennai from 1 to 10 October.
Embedding / vector	A list of numbers representing the meaning of a text, used to find similar texts.
Retrieval / recall	Finding the stored memories relevant to a question and returning them to the agent.
Projection	The copy of assertions and entities kept in Neo4j as a graph. It can be rebuilt from PostgreSQL.
Reflection	An optional step where the model writes an answer from retrieved memories, with citations.
Grounded answer	An answer in which every statement is backed by a cited, retrieved memory.
Evidence	Only in its ordinary sense: whatever is cited in support of a conclusion, such as the passages a reflected answer cites. A stored message is a source, not evidence.
Fine-tuning	Further training of an existing model on our own data so it does a specific task better.
Adapter	The small set of extra weights trained by LoRA/QLoRA; the base model itself stays frozen.
Quantisation	Storing model weights with fewer bits (e.g. 4-bit or 8-bit) so the model uses less memory and runs faster. Here "memory" does mean computer memory.
Teacher model	A larger model (GPT-5.6-sol) used to write the expected outputs for our training data.

Why “source” and “assertion”, not “evidence” and “fact”. Evidence is a role, not a kind of record: a message becomes evidence only when something cites it in support of a conclusion. A fact is something established, whereas our extracted units are produced by a model and can be wrong, contested, superseded or retracted. We therefore follow the knowledge-representation usage of claims and their sources. Wikidata stores each statement (claim) with references to the sources that back it [16], nanopublications keep an assertion separate from its provenance, the W3C PROV ontology links a derived entity to what it came from with `prov:wasDerivedFrom` and to a quoted passage with `prov:wasQuotedFrom` [17], and NELL holds extracted statements as “candidate beliefs” until enough support promotes them [18]. We also avoid “observation” for derived units, because Generative Agents [19] uses it for raw perceptions and “reflection” for derived ones (“reflection” already names question answering in our system). Zep/Graphiti [4], Mem0 [5] and Hindsight [7] call their extracted units “facts”; we deliberately depart from that usage.

CHAPTER 1

INTRODUCTION

AI agents are now used for coding projects that go on for weeks over many sessions, and every new session starts with an empty prompt. Whatever the user told the agent last week is simply gone unless the user types it in again. This project gives the agent a long-term memory outside the model.

1.1 BACKGROUND

The obvious fix is to paste the whole history into the prompt every time. This does not work well. Liu et al. [1] showed that language models use long inputs unevenly and miss details that sit in the middle of the context. Old statements also sit right next to their corrections with no rule for which one is current.

So a memory component has to decide what to keep and what to give back for a question. We built this as a separate service for a coding agent (Pi [9]). It stores the original messages, extracts assertions from them, remembers when an assertion changed, and when the agent asks, returns only the relevant assertions along with the quote they came from.

The word *memory* in this report always means this kind of stored knowledge about the user and the project, not RAM or any other computer memory. Other terms used in a specific sense, such as *source* (a stored message) and *assertion* (a statement extracted from it), are listed in the Glossary.

The engine has five modules: M1 saves the original messages as

source events (Source Acquisition and Processing), M2 extracts assertions from them (Knowledge Extraction and Consolidation), M3 stores the assertions with their history (Temporal Knowledge and Graph Management), M4 finds the right assertions for a question (Hybrid Retrieval and Grounded Reasoning) and M5 trains the small local model the others use (Model Training and Evaluation). M1 to M4 make up the memory service and M5 is an offline pipeline.

1.2 PROBLEM STATEMENT

Agents without persistent memory lose instructions and project decisions when a task continues across sessions, and when an instruction changes they may reuse the old value. Users end up repeating context and fixing the same mistakes again.

Throughout the report we use one small example, a user called Maya. On 1 October 2026 she tells the agent "My name is Maya. I live in Chennai. I work as a software engineer at NovaTech, and I work remotely." (we call this message E1). On 10 October she says she has moved to Bengaluru (E3). On 13 October she asks what the agent remembers about where she lives, her previous city and her work (E5).

An agent with no memory can't answer E5, and one that only kept E1 would confidently say Chennai, which is worse. We want it to know Bengaluru replaced Chennai on 10 October and still keep Chennai as her previous city.

1.3 OBJECTIVES

1. Retain useful information for AI agents across sessions.
2. Keep the updated instruction while preserving the earlier versions.

3. Link every stored assertion back to the original message so it can be verified.
4. Fine-tune a small local model to extract assertions, combine related assertions into inferences and answer with citations.
5. Retrieve relevant information quickly within a limited context size.
6. Evaluate memory accuracy and retrieval with repeatable tests on a public long-conversation benchmark, LoCoMo [15].

1.4 CHALLENGES FACED

On the fine-tuning side, our early models did well on generated test data but struggled on real conversations (only 18 of 33 conversation segments passed the extraction checks). The models also mixed up quotations, source references and dates. We revised the data, prompts and validation several times. One issue we have not solved: fine-tuning made combining assertions and answering questions much better, but extraction coverage dropped from 50/55 to 41/55.

On the integration side the model sometimes re-extracted an outdated value, so correction handling had to be reworked. Repeated messages made duplicates and one failed job could block the rest. The hardest bug to notice was that the deployed model silently cut off inputs longer than its limit, so now we check input length and split long conversations into smaller pieces.

1.5 ORGANISATION OF THE REPORT

Chapter 2 reviews existing memory systems, Chapter 3 the architecture and modules, Chapter 4 the tools and algorithms, and Chapter 5

the data, fine-tuning and demo, then the benchmark results, their analysis and the comparison with published memory systems.

CHAPTER 2

LITERATURE SURVEY

This chapter looks at existing work on long-term memory for agents. Every number quoted here is the one reported by the authors on their own setup; we did not rerun any of these systems.

2.1 LONG CONTEXT AND EXTERNAL MEMORY

The simplest memory is to keep the entire conversation in the prompt. Liu et al. [1] found that models use information in the middle of a long context less reliably than information near the start or end, so a bigger context window does not remove the need to choose what the model reads. It is also slow (17 s p95 latency for the full history in the Mem0 paper).

All the systems below handle this the same basic way. They keep memory outside the prompt and only put a selected part of it in front of the model. Where they differ is what they store, who decides how a stored item changes (the model or the code), and whether the old value of a changed item is kept.

2.2 MEMORY SYSTEMS

MemGPT. Packer et al. [3] split the prompt into a main context and external recall and archival storage, and let the model move data between them using function calls. On deep memory retrieval they report 92.5% for GPT-4 with MemGPT against 32.1% for GPT-4 reading a summary. A missed function call means a missed update.

Zep and Graphiti. Rasmussen et al. [4] store a temporal knowledge graph of episodes, entities and "facts" (their term). Each one has a valid time and an invalid time, and a contradicting one closes the old one instead of deleting it. They report 71.2% on LongMemEval with gpt-4o against 60.2% for the full 115k-token history, from a single run. This is closest to ours, except that in Zep the graph is the main store while we keep the real data in PostgreSQL.

Mem0. In Mem0 [5] an LLM extracts candidate "facts", compares each one with the nearest stored memories and picks one of add, update, delete or no-op. On LoCoMo they report a judge score of 66.9 against 72.9 for the full history, with p95 latency of 1.4 s against 17 s. Only one benchmark and an unnamed judge model were used, and the open-source release has since dropped update, delete and graph memory. We use the same extract-then-compare idea, but in our system the update decision is made by fixed code rules and not by the LLM for each extracted item.

Supermemory. Supermemory [6] stores atomic memories linked to source chunks, with "updates", "extends" and "derives" links and two timestamps on each. Its results are only from the vendor's own report and the engine is closed source.

Hindsight. Hindsight [7] is built around three operations, retain, recall and reflect. Recall combines four retrieval strategies using rank fusion and packs the result into a token budget. This directly influenced our recall step. It is not peer reviewed though.

Benchmarks. LongMemEval [2] tests chat assistants on long-term interactive memory, and Zep, Hindsight and Supermemory all report on it. LoCoMo [15] evaluates very long-term conversational memory and is the only benchmark used in the Mem0 paper. The published results aren't really

comparable since each uses its own models and judge.

2.3 METHODS USED IN THIS PROJECT

We use a few well known methods as building blocks. PostgreSQL full-text search [10] with GIN indexes is our keyword search. pgvector [11] adds vector columns and nearest-neighbour search to the same database, which we use with cosine distance on 1,024-dimensional vectors. Reciprocal Rank Fusion [12] merges several ranked lists by adding up $1/(k+rank)$ for each item; we use $k = 60$. The vectors come from a Qwen3 Embedding model [13] served through Ollama, and the memory model is fine-tuned with QLoRA [14], which trains small adapter weights on top of a frozen, quantised base model.

2.4 COMPARISON AND GAP

Table 2.1: Comparison of existing memory systems

System	Who decides an update	Main limitation
MemGPT	The model, through function calls	Edits depend on correct calls; no cost figures
Zep / Graphiti	Graph rule (validity intervals)	Single run; graph database is the main store
Mem0	The LLM, per item	One benchmark; released code differs from paper
Supermemory	Not fully documented	Closed source, vendor report only
Hindsight	Retain step	Not peer reviewed; budget not stated

Table 2.1 puts the five systems side by side. In all of them the model decides updates or the rule is tied to a graph database. So we took the extract-then-compare idea, the validity intervals and the fused recall, but made the update rule plain service code, kept PostgreSQL as the source of truth, tied every assertion to an exact quote, and trained a small local extraction model.

CHAPTER 3

SYSTEM DESIGN

This chapter describes the design requirements, the architecture and the five modules, following the Maya example through them.

3.1 DESIGN REQUIREMENTS

We worked from a few rules. Original messages are saved exactly as sent *before* any extraction, which runs later as a queued job, so a failed model call can't lose a conversation. A changed assertion is a new version and the old value is kept with its validity interval, never overwritten. Every assertion links to an exact source quote that is checked against the original message. All data belongs to a namespace (e.g. `user:maya`) and whatever we hand to a model or the agent has to fit a token budget. Models run locally on Ollama. Duplicates, crashed workers and bad model output are handled with stored jobs, leases, retries and validation, and Neo4j is only a copy that can be rebuilt from PostgreSQL.

3.2 OVERALL ARCHITECTURE

Figure 3.1 shows the overall architecture. The client is the Pi coding agent. Its memory client saves every message through the memory service and asks for recall before answering. The memory service itself is a Rust application with API handlers and asynchronous workers, and it depends on four other components.

PostgreSQL with pgvector is the main store, shared by M1 to M4. It holds the source events (sessions, episodes, events, chunks), the entities, assertions, source quotes and vectors, and the job queue. Ollama runs the models: our fine-tuned worker model for M2 (and reflection in M4), and a Qwen3 embedding model for vectors. Neo4j keeps a graph copy of the assertions and entities, joined by `MENTIONS` and `SUPPORTS` edges. M4 rechecks every graph hit against PostgreSQL before using it. M5 runs offline (live chats are *not* used as training data).

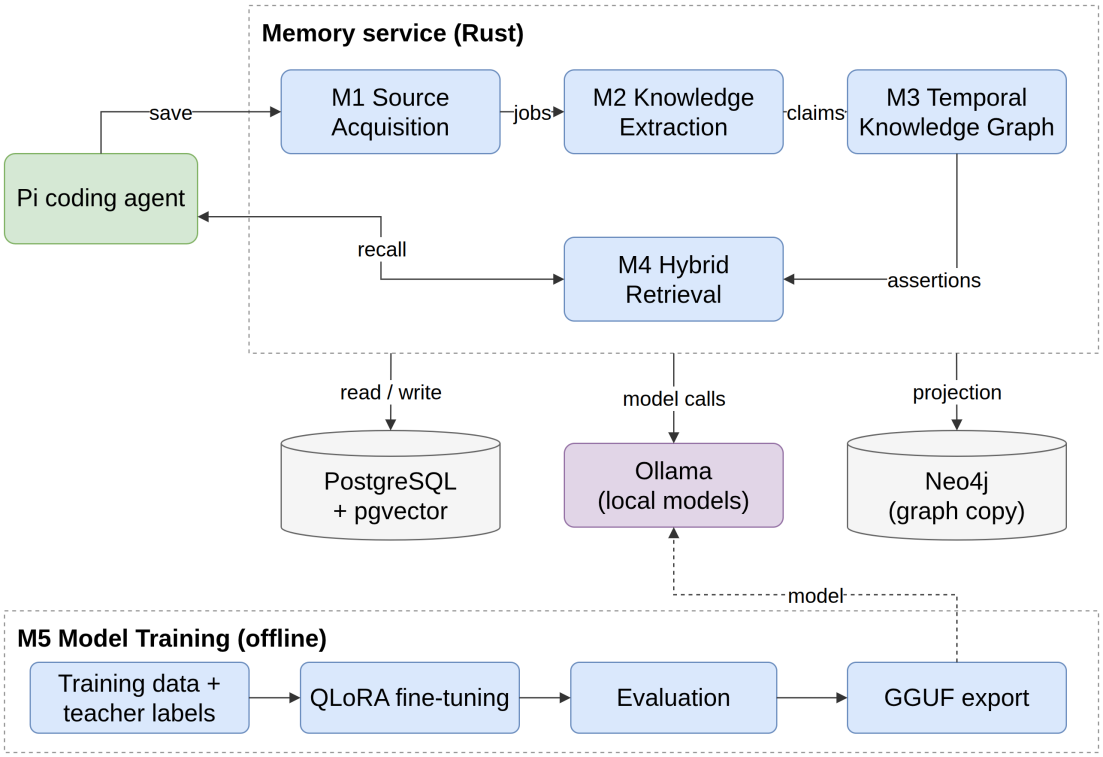


Figure 3.1: Overall system architecture

3.3 M1 SOURCE ACQUISITION AND PROCESSING

M1 saves the source events before anything else (Figure 3.2). For Maya, the 10 October conversation arrives as a retain request with namespace `user:maya`, session S2 and the two events E3 (her message) and E4 (the agent's reply).

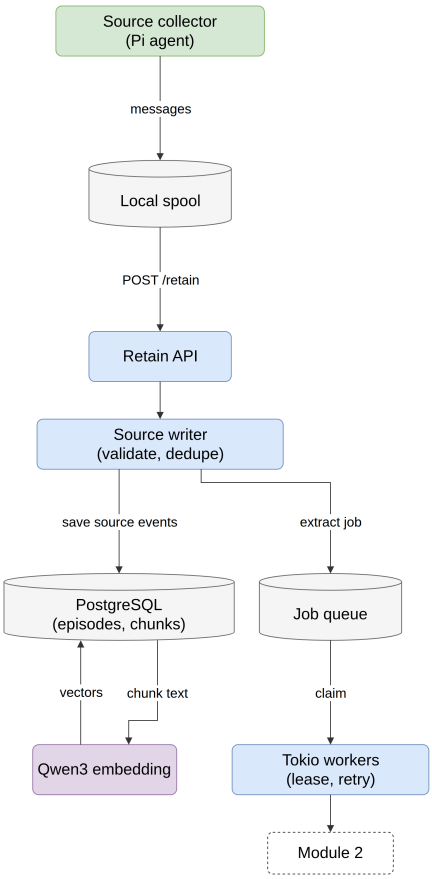


Figure 3.2: Module 1: Source Acquisition and Processing

The source collector inside Pi gathers the role, text and timestamp of each message on the current branch and forms a `RetainBatch`. This batch is first written to a local spool of JSON files so nothing is lost if the service is down, and the spool then sends it to `POST /api/v2/retain`. The writer checks the batch (1 to 1,000 events). If the same key arrives again with the same SHA-256 hash it just returns the existing IDs, so retries don't create duplicates; the same key with a different payload gets HTTP 409. The episode, events and chunks are saved in one transaction together with a pending extraction job in `memory_jobs`. Each chunk is also embedded into a 1,024-dimensional vector using `qwen3-embedding-cpu`.

Jobs are picked up by a pool of Tokio workers. A worker leases the namespace for 15 minutes (renewed every 30 s) and a job gets up to 5 tries with

backoff up to 300 s. We run two workers locally (the default is four). For Maya, M1 ends with episode P2 holding chunks C3 and C4, and job J2 pending.

3.4 M2 KNOWLEDGE EXTRACTION AND CONSOLIDATION

M2 has two stages, extraction and consolidation, and both call the same local model `mem-extractor:general-v1` through Ollama (Figure 3.3). Outputs are cached in `model_cache`.

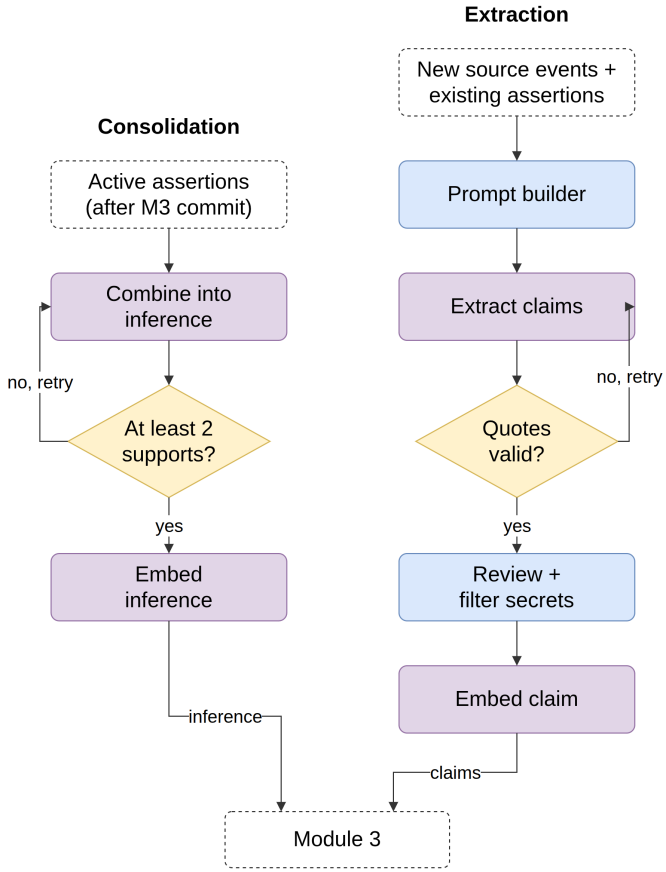


Figure 3.3: Module 2: Knowledge Extraction and Consolidation

Extraction. The prompt builder fits the new source events and the relevant existing assertions into the model's input (how existing assertions are chosen is in Section 4.2.1). For Maya the existing assertion "Maya / Chennai" is shown next to E3. The extractor returns claims with exact quotes. Here it returns

home city Bengaluru, single-valued and flagged as a correction. The output is checked against the schema and each quote must appear exactly in the source; if not, the model is asked again up to two times before the job fails. A review step confirms the correction, secrets are filtered out, and the claim “Maya lives in Bengaluru.” goes to M3.

Consolidation. After M3 commits, M2 looks at the active assertions of the changed subject. If there are at least two, the model may combine them into one inference. For Maya, the three work assertions A1 (NovaTech), A2 (software engineer) and A3 (remote) become “Maya works remotely as a software engineer at NovaTech.”

3.5 M3 TEMPORAL KNOWLEDGE AND GRAPH MANAGEMENT

M3 is the only module that writes assertions. Everything for one job happens inside a single PostgreSQL transaction that holds a namespace lock and checks the job’s lease (Figure 3.4).

For a claim, M3 first resolves the entity (Maya, a person in `user:maya`). If the same value already exists, it is just reinforced. If the slot is empty, a new active assertion is made. The interesting case is a different value in a single-valued slot, like Chennai to Bengaluru. If the claim is a reviewed correction with confidence at least 0.8, the old assertion is superseded and the new one becomes active; otherwise both are marked contested. Maya’s claim has confidence 0.95, so R1 Chennai becomes valid from 1 to 10 October and superseded, and R2 Bengaluru is active from 10 October, linked to the E3 quote.

After the transaction commits, a projection worker copies the

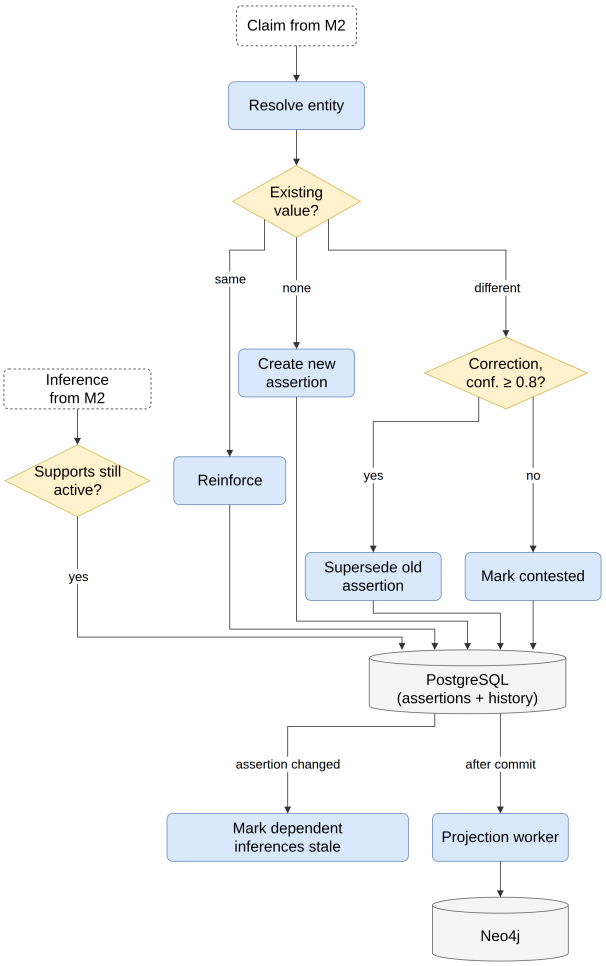


Figure 3.4: Module 3: Temporal Knowledge and Graph Management

changed assertion into Neo4j over HTTP [8]. These writes are revision-checked so an old revision can't overwrite a newer one.

3.6 M4 HYBRID RETRIEVAL AND GROUNDED REASONING

M4 answers recall requests (Figure 3.5). On 13 October Pi sends E5 with namespace `user:maya`, `top_k` 10, a maximum distance of 0.45 and a 2,048 token budget.

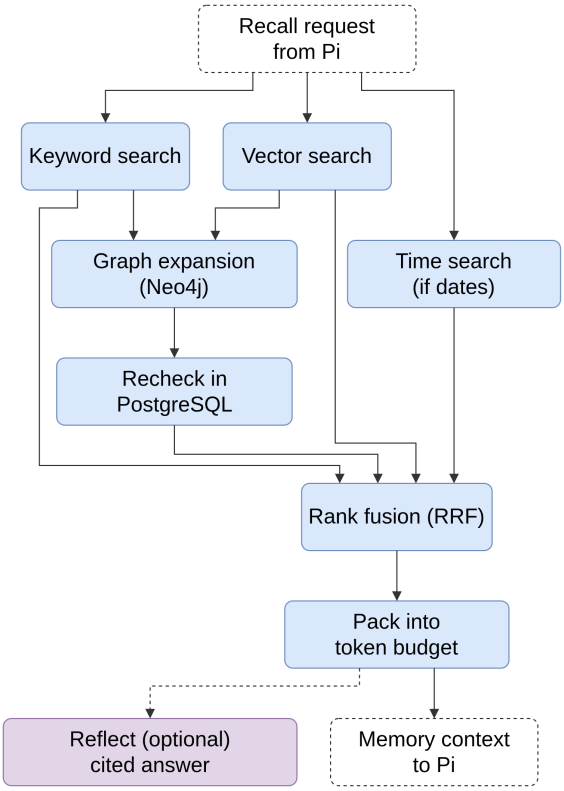


Figure 3.5: Module 4: Hybrid Retrieval and Grounded Reasoning

The query goes to keyword, vector and (if there's a date window) time-based search, the top results seed a Neo4j expansion, and the lists are merged with Reciprocal Rank Fusion and packed into the budget with up to three quotes per hit. There is also an optional reflect endpoint that writes a cited answer. All of this is in Chapter 4.

3.7 M5 MODEL TRAINING AND EVALUATION

M5 runs offline and produces the worker model used by M2 and M4 (Figure 3.6).

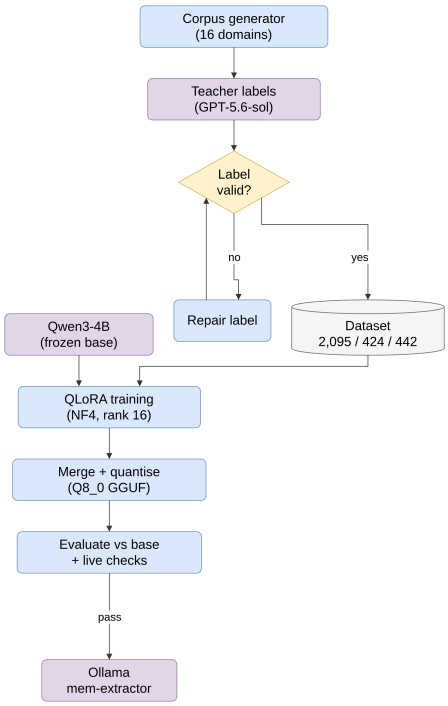


Figure 3.6: Module 5: Model Training and Evaluation

A corpus generator makes short episodes across 16 domains (the Maya episode is one of them) and a teacher model (GPT-5.6-sol) labels each one. Checked labels form the dataset, and the model is trained with QLoRA, evaluated and imported into Ollama (details in Chapter 5).

So when Maya asks E5, Pi answers “You now live in Bengaluru, following your move on 10 October 2026. Before that, you lived in Chennai. You work remotely as a software engineer at NovaTech.”

CHAPTER 4

IMPLEMENTATION

This chapter covers the tools we used, the six main algorithms (each with the Maya example walked through it) and how the engine plugs into Pi.

4.1 IMPLEMENTATION ENVIRONMENT

The memory service is written in Rust. Tokio gives us the async runtime for the background workers, Axum handles the API requests and SQLx talks to PostgreSQL. PostgreSQL 17 with pgvector is the main store and handles both full-text and vector search, and Neo4j 5.26 holds the graph copy used for graph-based retrieval.

The models run locally on Ollama. Our extraction model is a fine-tuned Qwen3-4B-Instruct-2507, which extracts assertions, combines them into inferences and writes cited answers. Qwen3-Embedding-0.6B makes the vectors. On the agent side we use the Pi agent harness to capture messages and inject memory. The dashboard (session controls and the memory graph) is React + TypeScript with Vite and Bootstrap.

Training used Python, PyTorch, Transformers, PEFT and bitsandbytes, with llama.cpp to make the GGUF. Docker Compose runs the services, and Caddy and ttyd give browser access to the app and the Pi console.

4.2 ALGORITHMS

Algorithm 4.1 Priority-based context selection

```
Input: existing assertions, new messages, input space

IF all existing assertions fit:
    RETURN all assertions
Score assertions by subject and value mentions in new messages
Rank stronger matches first; use recency to break ties
Use binary search to find the largest ranked subset that
    fits
RETURN selected assertions in their original order
```

There are six algorithms. Context selection is in M2; entity resolution and versioning is in M3; consolidation and invalidation is split between M2 (making inferences) and M3 (marking them stale); and retrieval, rank fusion with packing, and answer generation are all in M4.

4.2.1 Priority-Based Context Selection

The extraction model needs to see the assertions already stored about the same people, otherwise it can't tell a repeat from a correction, but they may not all fit its input limit. So assertions mentioned in the new messages are ranked first (recency breaks ties) and a binary search finds the largest prefix that fits.

For Maya, when E3 ("I moved from Chennai to Bengaluru today…") comes in, she has four stored assertions: Chennai, NovaTech, software engineer and remote. Assume for the example that only three fit. The home city assertion matches E3 on both subject and value, so it ranks first and the other three follow by recency. Binary search settles on three assertions, the home city one included, and these go to the extractor with E3.

4.2.2 Entity Resolution and Memory Versioning

Algorithm 4.2 Entity resolution and memory versioning

```
Input: a new claim and stored memories

Match entity by type and normalised name, or a unique
    stored alias; otherwise create an entity
IF the same assertion exists:
    Attach the new source quote
ELSE IF no conflicting single-value assertion exists:
    Store the claim as a new active assertion
ELSE IF explicit correction AND confidence >= 0.8:
    Supersede the old assertion; activate the new one
ELSE:
    Mark both values as disputed
```

This decides which entity a claim is about and what it does to the stored assertions. The important rule is that a conflicting value never overwrites silently, and the old assertion is superseded, not deleted, which is what lets us answer the "previous city" part of E5.

The E3 claim is Maya, home city Bengaluru, single-valued, `correction=true`, confidence 0.95, quote "Bengaluru is now my home city." Maya is found, so no new entity. The same assertion doesn't exist (R1 says Chennai), there is a conflicting single-valued assertion, and the claim is a correction with $0.95 \geq 0.8$. So R1 Chennai is superseded with validity 1–10 Oct, and R2 Bengaluru is active from 10 Oct with E3 as its source.

4.2.3 Memory Consolidation and Invalidation

The model proposes an inference but the code decides whether to save it. Supports are rechecked just before saving since they may have changed while the model was running. Invalidation goes the other way and marks dependent inferences stale.

Algorithm 4.3 Memory consolidation and invalidation

```
Input: active assertions about the affected entities

Ask the Extractor SLM to combine related assertions into
    inferences
Require at least two distinct supporting assertions
Check support IDs and the inference's subject
Recheck that supporting assertions are active before saving
Store each inference with its support links

WHEN a supporting assertion changes or is retracted:
    Recursively find dependent inferences
    Mark them as stale
```

For Maya the inputs are A1 (NovaTech), A2 (software engineer) and A3 (remote). The model proposes "Maya works remotely as a software engineer at NovaTech." with all three as supports. It has three distinct supports, the IDs exist, the subject is Maya, and all three are still active, so it is saved as I1. If A3 were retracted later, I1 would be found as a dependent and marked stale.

4.2.4 Hybrid Retrieval and Graph Traversal

This returns separate ranked candidate lists. The graph expansion is limited to two rounds so it doesn't pull in the whole graph. Merging is left to the next algorithm.

E5 has no date phrase, so there is no time-based list. Keyword and vector search find R2 (Bengaluru) and the work assertions. The graph step starts from these and the Maya entity: `MENTIONS` reaches Maya, and the `supersedes` link of R2 reaches R1 (Chennai). The graph hits are rechecked in PostgreSQL for status, time and distance, and the lexical, semantic and graph lists are passed on.

Algorithm 4.4 Hybrid retrieval and graph traversal

```
Input: a question and optional date restrictions

Resolve supported date phrases when planning is enabled
Find candidates using keyword and vector searches
Include time-based candidates when a date window exists
Start graph traversal from matching assertions and entities
Expand through entity/support links for two bounded rounds
Filter results by validity and any supplied relevance
    cutoff
RETURN candidate lists for ranking
```

Algorithm 4.5 Reciprocal Rank Fusion and context packing

```
Input: ranked candidate lists and a context budget

FOR each memory:
    Score = SUM[1 / (60 + rank)] across search lists
Sort by decreasing score; break ties by memory ID
FOR each ranked memory:
    Build a block with the assertion and its source quotes
    Add it if it fits; otherwise consider the next memory
    Stop when the requested number of memories is reached
RETURN the packed context
```

4.2.5 Reciprocal Rank Fusion and Context Packing

Scores from different searches can't be compared, so only ranks are used. A memory that several lists rank highly beats one that a single list ranks first, and ties go by ID so the order is deterministic.

To illustrate, say R2 is ranked first and R1 second in the lexical, semantic and graph lists. Then R2 scores $3 \times 1/61 \approx 0.0492$ and R1 scores $3 \times 1/62 \approx 0.0484$, so R2 comes first and no tie-break is needed. With Pi's settings (`top_k` 10, 2,048 tokens) both blocks fit easily, and R2 goes out with

Algorithm 4.6 Source-grounded answer generation

```
Input: a question and retrieved memories

IF no memories were retrieved:
    RETURN "Insufficient evidence."
Ask the extractor SLM to answer from the retrieved memories
Restrict citation IDs to the retrieved memories
Check the response format and citation IDs
Require citations when sufficient evidence is claimed
RETURN the answer, or reject invalid output
```

the E3 quote attached.

4.2.6 Source-Grounded Answer Generation

This decides whether the model's answer is accepted. Bad output is rejected rather than patched up, so every citation the user sees points to a memory that was actually retrieved.

For E5 the retrieved memories are R2, R1 and the work assertions, so the model is called. Only R1, R2 and the work assertion IDs are allowed as citations; the answer cites R2 for Bengaluru and R1 for Chennai, so it passes.

4.3 INTEGRATION WITH THE PI AGENT

Pi's memory client saves every message, tagged with a namespace and session. When Maya asks E5, Pi sends a recall request for `user:maya` with `include_raw` true, `max_distance` 0.45, `top_k` 10 and `max_tokens` 2,048. The result is injected only if non-empty and from the same namespace.

CHAPTER 5

RESULTS, PERFORMANCE ANALYSIS AND COMPARISON

This chapter describes the training data and the fine-tuned model, shows the engine in a live session, and then measures it. The measurement asks one question: does an agent answer questions about a long conversation better with the memory engine than without it, and at what cost? It ends with a comparison against published memory systems and the limits of what the results show.

5.1 DATASETS

Each training row has the same shape: the input is a short conversation plus a task (extract, consolidate or reflect), and the output is the expected memory, i.e. assertions, inferences or an answer, each tied to a source quote. For example in the Maya episode the user says "I live in Chennai." and later "Bengaluru is now my home city.", and the expected output is the assertion "Maya, home city, Bengaluru" with `correction:true`, `source:user` and that quote. The training rows and prompts keep the wording the model was trained on (kinds `fact` and `observation`, the heading "Existing facts:"); the service translates these to `attribute` and `inference` at the model boundary, so the renaming used in this report does not change the model's contract.

The fine-tuning data started out as only coding conversations and we later expanded it to 16 domains (family, food, travel, education, work, software and so on). All conversations are synthetic. They were generated by the teacher

model GPT-5.6-sol, cross-checked by a second adversarial model and then cleaned. In total there are 2,961 task examples, split into 2,095 for training, 424 for validation and 442 for testing. Figure 5.1 shows the spread across domains. Software and technical projects is the biggest with 240 rows and the rest have roughly 110 to 140 each. (Several tasks can come from one episode, so this counts rows, not conversations.)

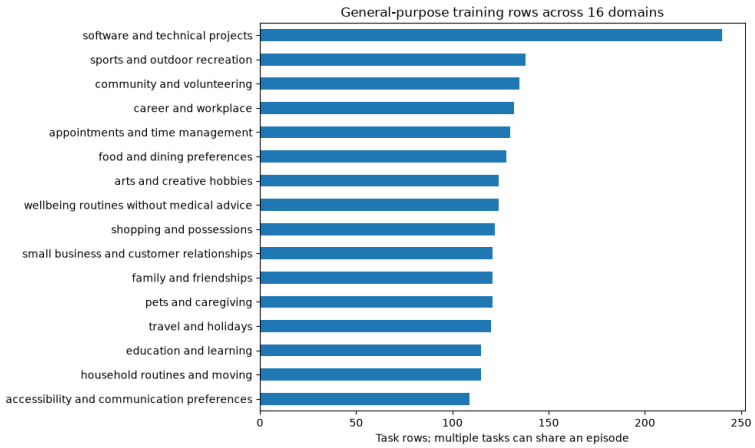


Figure 5.1: General-purpose training rows across the 16 domains

A lot of the rows test whether the model keeps only what the user actually said. For example, the assistant says "You could learn violin this winter." and the user replies "No, that is not my plan. I am learning watercolor painting." The expected memory is just that the user is learning watercolor painting. The assistant's suggestion should not become a memory.

The benchmark in Section 5.4 showed that this data asked for too few facts per conversation: most episodes expected two or three claims, so the model learned to stop early. A third set was therefore generated for coverage: 480 casual conversations between two people, each labelled with every durable fact it contains by Claude Sonnet 5.5 as teacher (5,284 claims; 25 of the 480 labels failed validation and were dropped). Together with 600 rows replayed from the earlier stages so that consolidation and answering were not forgotten,

this gave 1,719 training and 670 validation rows. None of it uses LoCoMo text, so the benchmark below stays unseen.

5.2 FINE-TUNING AND MODEL SELECTION

The model is Qwen3-4B-Instruct-2507, fine-tuned with QLoRA [14] and served through Ollama. Following Figure 3.6, GPT-5.6-sol labelled the episodes for the extract, consolidate and reflect tasks, every label was checked for structure and source quotes (and repaired if needed), and the valid ones were frozen into the 2,095/424/442 split. Rows were tokenised with a 4,096 token limit and the prompt was masked so only the target is supervised. Training used QLoRA on the frozen base with NF4 quantisation and rank 16, going from Stage A to Stage B. The adapter was then merged and quantised to a Q8_0 GGUF with llama.cpp, compared against the base model on 64 internal tasks and 16 external cases, checked live (retain, graph, recall/reflect), and packaged with a SHA-256 checksum and imported into Ollama as `mem-extractor:general-v1`.

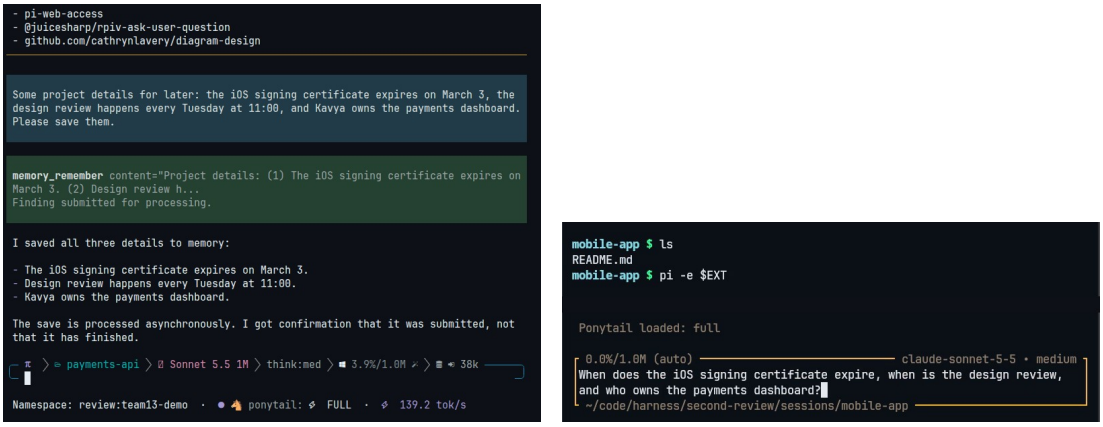
We actually started with Qwen3-1.7B. It was fine on synthetic examples but bad on natural conversations, only 18 of 33 extraction windows passed validation even after refinement. Most failures were wrong source attribution, quotes that didn't match and invalid IDs. That's what pushed us to broaden the training data, tighten the output constraints and move up to the 4B model.

Stage C continued from the Stage B adapter on the coverage set (one epoch, learning rate 3×10^{-5}, rank 16, gradient accumulation 8). The limit per row was raised to 6,144 tokens because the new rows, which list every fact, are longer. Training took 2.9 hours on the laptop's 8 GB RTX 4070 (peak memory 7.3 GB, checkpoints every 20 steps so that an out-of-memory crash resumes rather than restarts); the validation loss was 0.155. The result was exported in

the same way as `mem-extractor:general-v2`. Which worker to use for the test was decided by a rule written before the comparison: `general-v2` replaces `general-v1` if its accuracy on the two development conversations is at least as high. It was (Section 5.5).

5.3 LIVE DEMONSTRATION

The demo follows the same idea as Maya (record, ask later in a new session, see the correction) but with its own values: a certificate expiry date, a weekly review time, a dashboard owner, and later a release-freeze time. Figure 5.2 shows the first two steps.



(a) Recording in the first session (b) Asking in a new session

Figure 5.2: (a) The user gives a certificate expiry date, a weekly review time and the dashboard owner, which are submitted for memory processing. (b) A fresh session in another project directory asks for the same details without repeating them

In (a) the agent calls the `memory_remember` tool and notes that processing is asynchronous, so it can't confirm storage yet. In (b) a new session in `mobile-app` (not `payments-api`) asks for all three without restating them, like Maya asking E5.



(a) Recalled answer (b) Corrected assertion in the graph

Figure 5.3: (a) Memory from the first session is added to the context and the agent answers March 3, Tuesday at 11:00 and Kavya. (b) The graph shows Friday at 16:30 as the current release-freeze time, linked to the previous Thursday at 17:00 value, with the supporting message and history in the side panel

In Figure 5.3(a) the agent answers from the recalled assertions (March 3, since no year was given, then Tuesday 11:00 and Kavya) and says the information came from an earlier session. It also notes that the extracted assertion stored the date as 2026-03-03 although the user gave no year, which shows why answers cite their source quotes. In (b) the Friday 16:30 freeze is linked to the old Thursday 17:00 value by `contradicts` and `supersedes`, same as Bengaluru and Chennai.

5.4 EXPERIMENTAL SETUP

The benchmark asks one question: does an agent answer questions about a long conversation better with the memory engine than without it, and at what cost? It uses LoCoMo [15], the dataset on which Mem0, Zep and A-Mem report their results [5]. LoCoMo has ten long conversations between

two people, 272 sessions in all (about 16,000 tokens per conversation), and 1,540 questions with gold answers in four categories: single-hop, multi-hop, temporal and open-domain. The 446 adversarial questions have no gold answer and are left out, as in the Mem0 evaluation. Each conversation is retained into the engine as its own memory bank, session by session in date order. Questions, answers and evidence labels are never ingested.

5.4.1 Conditions

Every question is answered by the same reader model with the same prompt under each condition in Table 5.1; only the text the reader is given changes.

Table 5.1: Benchmark conditions

Condition	What the reader is given
`enhanced`	The engine's recall: assertions with source quotes, inferences, graph neighbours and temporal planning, interleaved with source text, packed into 8,192 tokens
`raw_hybrid`	Lexical and vector search over the raw source chunks only (plain retrieval-augmented generation), same budget
`recent_window`	The newest 8,192 tokens of the conversation: what an agent without memory keeps when history overflows
`full_context`	The whole conversation: an upper reference and the most expensive option
`no_memory`	The question only
`assertions_only`, `no_graph`, `no_consolidation`	`enhanced` without source text, the graph or inferences, on a 300-question sample stratified by category

5.4.2 Reader, judges and metrics

The reader is Claude Haiku 5.5, called through the Pi coding agent, with Mem0's single-store answer prompt plus one line asking for the final answer as JSON. Two judges from different model families grade each answer against the gold answer with Mem0's LoCoMo rubric, and never see which condition produced it: TypeSafe Jev [20], a decision model that returns the probability that the answer is correct (the primary judge, threshold 0.5), and GPT-4o-mini, called exactly as in Mem0's released evaluation code so the numbers can be compared with published tables. Mem0's token F1 and BLEU-1 are computed without any model. Retrieval is scored against LoCoMo's evidence labels (the dialogue turns that answer each question), and every source quote a recall returns is checked against the dataset turn it cites.

Differences are reported as paired bootstrap intervals [21] (2,000 resamples), both over questions and over whole conversations (a cluster bootstrap), because the questions about one conversation share one memory bank. Two extra repetitions on the 300-question sample measure how much the reader's answers vary between runs. The agreement of the two judges is reported as Cohen's kappa [22].

5.4.3 Protocol

The conditions and success criteria were committed to the repository before the test run, and the run's manifest records the commit together with a fingerprint of all code and configuration; a test run refuses to continue if either changes. Faults were diagnosed only on two conversations (the dev split), and every change made because of them is recorded with its before and after numbers. The test run uses all ten conversations with the final version, and the eight conversations never used for diagnosis are also reported on their own.

5.4.4 Reproducibility

The benchmark is one command (`benchmark/overnight.sh`) on a single laptop with an 8 GB RTX 4070, and takes about seven hours, most of it building memory. The dataset, the vendored Mem0 prompts and the fine-tuned model are pinned by SHA-256. Every reader and judge call is stored with its full output and the hash of its prompt. The `verify` command rebuilds each stored prompt from the stored retrieval and the pinned dataset and fails if any stored answer has been changed; with `-live` it also re-asks the models on random items. The executed notebook `Benchmark_Report.ipynb` recomputes every number in this chapter from the stored records and prints random items in full for checking by hand.

A separate property suite of 40 scripted conversations checks the behaviours claimed for the engine on its stored state, without a judge: a knowledge update keeps the old value as history and `as_of` recall returns it, a relative day such as "yesterday" is resolved to a date, the engine abstains without evidence, a relative's move does not overwrite the user's own city, and a credential is never stored.

5.5 DEVELOPMENT ITERATIONS

The first full runs on the two development conversations exposed faults in the engine, the extractor and the benchmark harness. Each was diagnosed on those two conversations only, fixed, and recorded in `benchmark/iterations.json` with the evidence before and after. Table 5.2 summarises them.

Table 5.2: Changes made because of development-run findings

Version	Finding	Change and effect
v1	A crowded session made the extractor's answer exceed its 3,072-token output limit, so the bank could not be built	A window whose answer is cut off is split in two and retried; the bank was built
v2	"I went to a support group yesterday", said on 8 May, was stored as 8 May	A relative day named in the claim's own quote sets the event date
v3	After about a dozen sessions, consolidation prompts for one speaker exceeded the model's context and every such job failed (18 of 28)	The newest assertions that fit are kept and the rest left out, with a warning; all 38 succeed
v4	The review step vetoed explicit corrections ("I moved from Hobart to Accra"), and some "in three days" dates were miscounted	A correction whose quote states the change is kept; miscounted days are corrected from the quote
v5	Recall built from assertions alone answered 45.9% against 79.8% for plain retrieval, because the extractor missed many facts	Source text is interleaved with assertions in recall (top 60); 82.8% against 82.4% for plain retrieval
v6	The extractor's training episodes asked for two or three claims each, so it under-extracted	Stage C training on the coverage set; assertions alone 51.9% $\rightarrow$ 63.1%, the engine 82.4% $\rightarrow$ 85.0%

Three further faults were in the harness rather than the engine: evidence labels used a different session name from the one recall reports, so retrieval scored zero; an engine restart crashed the ingest stage instead of being waited out; and the jobs API listed only the newest 100 jobs, which hid failed consolidations behind hundreds of newer projection jobs. All three were fixed before the test run. Accuracies in Table 5.2 are on the 233 questions of the two development conversations and should be read with the run-to-run noise measured in Section 5.6: plain retrieval, which does not use the extractor at all, moved from 81.1% to 83.7% between the two v6 runs.

5.6 RESULTS

The test run (`locomo-final`, commit `fb044f2`) built a memory bank for each of the ten LoCoMo conversations and answered all 1,540 questions under each condition. Table 5.3 gives the verdict on the criteria fixed before the run: the memory engine passes all of them, on all ten conversations and on the eight never used for diagnosis.

Table 5.3: Success criteria fixed before the test run (Jev)

Criterion	**Threshold**	**All ten**	**Held-out eight**
Memory engine beats no memory	by $\geq$ 30 pts	pass	pass
Memory engine beats the recent window	conversation CI above 0	pass	pass
Memory engine not below plain retrieval	conversation CI not wholly below 0	pass	pass
Memory engine above plain retrieval (not required)	conversation CI above 0	pass	pass
Close to the whole conversation	within 10 pts	pass	pass
Uses at most half its tokens	share $\leq$ 0.5 (got 0.30)	pass	pass
Every source quote exact	100%	pass	pass

Table 5.4 gives the full results. With the engine the reader answered 83.1% of the questions correctly. Without any memory it answered almost none (0.3%), because LoCoMo asks about details of the conversation that no model can know. An agent that keeps only its most recent 8,192 tokens, which is what happens when a long history overflows the context window, answered 43.0%: the evidence for most questions lies in earlier sessions. Plain retrieval over the raw text answered 80.0%, and giving the reader the whole conversation, about 22,400 tokens, answered 89.0%. The engine therefore closes most of the gap between plain retrieval and the whole conversation while giving the reader 30% of the tokens.

Table 5.4: LoCoMo results by condition (all ten conversations, categories 1–4)

Condition	n	Jev	GPT-4o-mini	F1	BLEU-1	Tokens	Reader s
Memory engine	1540	83.1	83.2	55.8	48.4	6,790	5.2
Plain retrieval	1540	80.0	80.1	53.5	46.4	4,473	5.5
Recent window	1540	43.0	42.3	28.9	25.0	8,168	5.2
Whole conversation	1540	89.0	88.8	60.4	52.6	22,400	5.5
No memory	1540	0.3	0.4	0.5	2.0	7	4.1
Assertions only	300	66.7	66.7	41.3	35.7	5,566	5.2
Without graph	300	76.3	76.7	47.9	41.6	6,606	5.3
Without consolidation	300	74.7	75.7	47.3	40.5	6,635	5.1

Jev and GPT-4o-mini: percentage of answers judged correct against the gold answer with Mem0's LoCoMo rubric. F1 and BLEU-1: Mem0's token overlap with the gold answer. Context tokens: mean tokens given to the reader besides the question. Reader s: median seconds per answer. The last three rows are on the 300-question stratified sample.

Table 5.5 gives the paired differences. The engine is 3.1 points above plain retrieval; the 95% interval is [1.2, 4.8] over questions and [2.0, 4.0] over conversations, so the gain is small but not noise. It is 6.0 points below the whole conversation and 40.1 points above the recent window. GPT-4o-mini, the second judge, gives the same ordering and nearly the same differences (Table 5.6): 3.1 points above plain retrieval, interval [1.2, 5.0] over questions.

Table 5.5: Accuracy of the memory engine minus each other condition, same questions (Jev)

Compared with	n	Difference (pts)	95% CI, questions	95% CI, conversations
Plain retrieval	1540	+3.1	[+1.2, +4.8]	[+2.0, +4.0]
Recent window	1540	+40.1	[+37.3, +42.7]	[+36.3, +43.5]
Whole conversation	1540	-6.0	[-7.9, -4.3]	[-6.9, -5.0]
No memory	1540	+82.8	[+80.8, +84.6]	[+80.8, +84.9]
Assertions only	300	+9.7	[+4.7, +14.7]	[+4.1, +15.1]
Without graph	300	+0.0	[-3.7, +3.7]	[-4.3, +3.6]
Without consolidation	300	+1.7	[-1.3, +4.3]	[+0.3, +3.3]

Paired bootstrap with 2,000 resamples, over questions and over whole conversations (cluster bootstrap). A positive difference means the memory engine answered more questions correctly.

Table 5.6: Accuracy of the memory engine minus each other condition, same questions (GPT-4o-mini)

Compared with	n	Difference (pts)	95% CI, questions	95% CI, conversations
Plain retrieval	1540	+3.1	[+1.2, +5.0]	[+1.4, +4.6]
Recent window	1540	+40.8	[+38.1, +43.5]	[+37.3, +44.2]
Whole conversation	1540	-5.6	[-7.5, -4.0]	[-7.2, -4.1]
No memory	1540	+82.8	[+80.8, +84.6]	[+80.5, +85.1]
Assertions only	300	+9.3	[+4.3, +14.0]	[+4.1, +16.5]
Without graph	300	-0.7	[-4.0, +2.7]	[-3.4, +1.6]
Without consolidation	300	+0.3	[-2.7, +3.0]	[-1.8, +2.0]

Paired bootstrap with 2,000 resamples, over questions and over whole conversations (cluster bootstrap). A positive difference means the memory engine answered more questions correctly.

5.6.1 Held-out conversations

All faults in Section 5.5 were found on conversations 26 and 30, so a gain that came from tuning to those two would shrink on the rest. Table 5.7

shows it does not: on the other eight the engine answers 83.0% against 79.4% for plain retrieval and 88.9% for the whole conversation, the same picture as on all ten.

Table 5.7: All ten conversations against the eight never used for diagnosis

Condition	Jev, all ten	Jev, held-out eight	GPT-4o-mini, held-out eight
Memory engine	83.1	83.0	82.8
Plain retrieval	80.0	79.4	79.0
Recent window	43.0	41.5	40.6
Whole conversation	89.0	88.9	88.3
No memory	0.3	0.2	0.4

Held-out conversations: conv-41, conv-42, conv-43, conv-44, conv-47, conv-48, conv-49, conv-50.

5.6.2 Question categories

Table 5.8 splits the results by category. The engine's advantage over plain retrieval comes mostly from multi-hop questions (83.0% against 74.5%), which need facts from several sessions. Asked "What subject have Caroline and Melanie both painted?", the reader answered "Nature" from plain retrieval's chunks; the engine's recall held one assertion per person ("Caroline painted a sunset scene ..." and "Melanie painted a sunset with a palm tree ..."), and the reader answered "Sunsets". An assertion states in one sentence what the conversation spread over several turns, so facts from different sessions fit side by side in the budget. On single-hop questions the engine is 3.1 points ahead. On temporal and open-domain questions the two are level (80.1% against 80.4%, and 58.3% against 60.4% on only 96 questions). Temporal questions are where the engine should help most and did not; Section 5.8 shows why.

Table 5.8: Accuracy by LoCoMo question category (Jev, %)

Category	n	Memory engine	Plain retrieval	Recent window	Whole conversation	No memory
Single-hop	841	87.0	83.9	44.9	93.3	0.0
Multi-hop	282	83.0	74.5	51.4	88.7	0.4
Temporal	321	80.1	80.4	31.2	85.7	0.0
Open-domain	96	58.3	60.4	40.6	63.5	3.1

5.6.3 Ablations

On the 300-question sample (75 per category), removing the source text from recall (`assertions_only`) costs 9.7 points (interval [4.7, 14.7] over questions), so extracted assertions do not yet replace the text they came from; they help when given beside it. Removing consolidation costs 1.7 points by Jev and 0.3 by GPT-4o-mini, and removing the graph 0.0 and 0.7; none of these intervals over questions excludes zero (for consolidation by Jev, $[-1.3, 4.3]$), so on this sample neither module has a measurable effect on accuracy. LoCoMo's questions are about two people and rarely need a chain of relations, so this benchmark cannot show what the graph is for; its role in tracing corrections is checked by the property suite instead.

5.6.4 Run-to-run variation and judge agreement

The reader does not give the same answer every time. On the 300-question sample each condition was asked three times (Table 5.9). The engine's accuracy varied by 0.5 points (standard deviation) between runs and plain retrieval's by 1.3, while 7 to 8% of individual verdicts changed in at least one run. Differences of a point or two on a few hundred questions are therefore within noise, which is why the development-run changes in Section 5.5 are

read with caution; the 3.1-point gain over plain retrieval on 1,540 questions is outside it. The two judges agree closely (Table 5.10): they give the same verdict on 96.7% of the 8,600 answers (Cohen's $\kappa = 0.93$), and no condition's accuracy differs between them by more than 1.0 point.

Table 5.9: Run-to-run variation: the reader asked three times on the 300-question sample (Jev)

Condition	n	Accuracy, runs 0/1/2 (%)	SD (pts)	Verdicts changed (%)
Memory engine	300	76.3, 77.0, 77.3	0.5	7.3
Whole conversation	300	85.0, 85.7, 86.0	0.5	7.7
Plain retrieval	300	76.3, 74.3, 76.7	1.3	8.0
Recent window	300	47.3, 47.7, 46.3	0.7	6.7

Verdicts changed: share of questions whose verdict differs in at least one repeat.

Table 5.10: Agreement of the two judges

Answers	n	Same verdict (%)	Cohen's κ
All graded answers, every condition	8,600	96.7	0.932

5.7 PERFORMANCE ANALYSIS

5.7.1 Retrieval and provenance

Table 5.11 scores retrieval against LoCoMo's evidence labels, the dialogue turns that answer each question. The engine and plain retrieval find the right session about equally often (80% in the top five, about 97% somewhere in the packed context), but the engine's context contains the exact evidence turn more often (84.4% against 80.6%), because an assertion carries the quote of the turn it came from even when its own text ranks the turn low. Assertions alone reach the evidence turn for only 62.2% of questions, which matches their lower accuracy: the extractor still misses facts. Every one of the 99,971 source quotes

the engine returned matched the dataset turn it cites exactly, so an answer built from recall can always be traced to what was said.

Table 5.11: Retrieval against LoCoMo's evidence labels (%)

Measure	Memory engine	Plain retrieval	Assertions only
Evidence session in top 5	79.9	80.3	51.1
Evidence session in top 10	88.4	89.5	74.7
All evidence sessions in top 10	83.5	84.4	66.1
Evidence session in the packed context	96.9	97.6	93.4
Evidence turn in the packed context	84.4	80.6	62.2
Source quotes checked	99,971	61,803	20,386
Quotes exactly matching the cited turn	100.00	100.00	100.00
Recall latency, ms (p50 / p95)	428 / 584	420 / 569	422 / 541

5.7.2 Tokens and latency

The engine gives the reader 6,790 tokens of context per question on average, 30% of the 22,400 of the whole conversation; counting the reader's prompt and answer, a question costs 14,900 tokens against 35,300. For LoCoMo's conversations, about 22,000 tokens each, the whole conversation still fits in Haiku's context window, and the reader answers it about as fast (5.5 s against 5.2 s at the median) because a hosted model reads input in parallel. The engine's advantage here is therefore cost and the ability to grow: its context stays within the 8,192-token budget however long the history becomes, while the whole conversation grows with every session and eventually does not fit. The engine's own recall takes 428 ms at the median and 584 ms at the 95th percentile, including the query embedding, so it adds less than a tenth to the reader's time.

5.7.3 Cost of building memory

Building memory is the expensive step (Table 5.12). The ten conversations, 272 sessions and 5,882 turns, took 4.2 hours of wall-clock time on the laptop, four conversations at a time: about 56 seconds of wall-clock time per session, nearly all of it extraction by the 4B model. No job failed permanently. This cost is paid once, in the background, when a session is retained; it does not delay the agent's answers. A larger GPU or a second model slot would shorten it in proportion.

Table 5.12: Cost of building memory for LoCoMo on one RTX 4070 laptop GPU (8 GB)

Measure	**Value**
Conversations / sessions / turns	10 / 272 / 5,882
Characters retained	858,994
Bank build time, summed over conversations	15.9 h
Model time in extraction	15.2 h
Model time in consolidation	0.63 h
Jobs that failed permanently	0

Four conversations were built at a time, so summed build time exceeds wall-clock time.

5.8 PROPERTY SUITE

LoCoMo's questions do not separate a current value from an old one, and its judge cannot see the stored state. The property suite checks those behaviours directly on 40 scripted conversations (Table 5.13). Abstention, credential handling and knowledge updates behave as designed: the engine declines to answer without evidence in all six cases, never stores an API key, and after each of the 12 updates returns the new value first while `as_of` recall returns the old one. Every source quote is verbatim.

The suite also found a fault the LoCoMo judge could not see. In the test run no relative date was resolved (0 of 12): the Stage C worker writes

the resolved date into the statement, “Priya plans to run a half marathon on 2025-09-24”, but often leaves the event date empty, and the store then filled it with the time the message was sent. The reader still answered all 12 date questions correctly from the statement, which is why accuracy did not reveal it. The engine now resolves a relative day named in the claim’s quote when the event date is empty as well (commit `43af5bd`); re-run on the fixed engine (`properties-v7`), all 10 extracted events carry the right date, and the other two were never extracted. The LoCoMo numbers above are for the engine before this fix.

The remaining failures are all facts the extractor did not produce, not facts it got wrong. When a relative moves, the move is always attributed to the relative and never overwrites the user’s city, but in four of six conversations the user’s own city was never extracted; in two of four credential conversations, the ordinary fact said in the same message as the API key was dropped together with the key; and in two of 12 updates the old value was not stored as history. This is the same coverage gap the `assertions_only` ablation shows.

Table 5.13: Property suite: checks on the engine's stored state, no judge

Family	Check	Test run	After fix
abstention	abstains without evidence	6/6	6/6
abstention	answers with evidence	6/6	6/6
abstention	source quotes verbatim	6/6	6/6
attribution	own city not superseded	6/6	6/6
attribution	own city still active	2/6	2/6
attribution	relative move not attributed to user	6/6	6/6
attribution	source quotes verbatim	6/6	6/6
credential	ordinary fact kept	2/4	2/4
credential	secret never stored	4/4	4/4
credential	source quotes verbatim	4/4	4/4
knowledge update	as of recall returns old value	12/12	12/12
knowledge update	new value active	12/12	12/12
knowledge update	old value kept as history	10/12	10/12
knowledge update	old value not current	12/12	12/12
knowledge update	recall ranks current value first	12/12	12/12
knowledge update	source quotes verbatim	12/12	12/12
relative date	event extracted	10/12	10/12
relative date	relative date resolved	0/12	10/12
relative date	source quotes verbatim	12/12	12/12

5.9 COMPARISON WITH EXISTING SYSTEMS

Table 5.14 places the result beside the LoCoMo results published with Mem0 [5], which also reports Zep [4], LangMem [23], A-Mem [24], OpenAI's memory and a tuned retrieval baseline, all graded by GPT-4o-mini with the rubric used here. The absolute scores cannot be compared directly: the published systems answer with GPT-4o-mini and this work with Claude Haiku 5.5, a stronger reader, which lifts every condition, including the whole-conversation baseline (88.8% here against 72.9% there). The fairer comparison is each system's distance from the whole-conversation baseline of its own study, the last column. There the engine is 5.6 points behind, against

4.5 points for Mem0 with its graph, 6.0 for Mem0, 6.9 for Zep and 11.9 for the best retrieval baseline; this work's own plain retrieval is 8.8 points behind. By this measure the engine is in the same range as Mem0 and Zep, not ahead of them, and it gives the reader more context to get there (6,790 tokens against 1,764 for Mem0 and 3,911 for Zep). A stronger reader also leaves less room for memory to help, so the distances are only a rough guide.

Table 5.14: Published LoCoMo results [5] beside this work (GPT-4o-mini judge)

System	J (%)	Context tokens	J minus own full context
Full-context	72.90	26,031	+0.00
Mem0g	68.44	3,616	-4.46
Mem0	66.88	1,764	-6.02
Zep	65.99	3,911	-6.91
RAG (best, k=2, 256-token chunks)	60.97	–	-11.93
LangMem	58.10	127	-14.80
OpenAI	52.90	4,437	-20.00
A-Mem	48.38	2,520	-24.52
This work: whole conversation (Haiku 5.5)	88.83	22,400	+0.00
This work: memory engine (Haiku 5.5)	83.18	6,790	-5.65
This work: plain retrieval (Haiku 5.5)	80.06	4,473	-8.77

Published systems answer with GPT-4o-mini; this work answers with Claude Haiku 5.5, so absolute scores are not comparable. The last column compares each system with the full-context baseline of its own study. Context tokens for published systems are from the Mem0 paper.

Beyond accuracy, the engine differs from these systems in what it keeps. Mem0 and A-Mem rewrite or delete a memory when it is updated; the engine keeps every version with its validity interval and source quote, which is what lets `as_of` recall answer about the past and lets every answer be traced to a turn of the conversation. It also runs entirely on one laptop: extraction, consolidation and embedding use a 4B model on an 8 GB GPU, where the

published evaluation called a hosted model for every memory operation.

5.10 THREATS TO VALIDITY

- **One benchmark.** LoCoMo has ten conversations between two people; LongMemEval was planned and not run. Results on other kinds of history, such as an agent's coding sessions, are not measured here.
- **Few conversations.** Intervals over conversations rest on ten clusters. With so few, a bootstrap interval can be too narrow, and here it was sometimes narrower than the interval over questions, so conclusions are drawn only where both intervals agree.
- **Development on the test set.** Faults were diagnosed on two of the ten conversations, and the test includes them. The eight held-out conversations give the same result.
- **Model judges.** Accuracy is decided by two model judges, not people. They agree on 96.7% of answers (Cohen's κ 0.93), and the notebook prints random items in full for checking by hand.
- **Reader.** One reader model was used. A weaker reader may rely more on the memory, a stronger one less.
- **Equal budgets.** The engine and plain retrieval share the 8,192-token budget, but plain retrieval used 4,473 tokens of it against 6,790 for the engine, because its chunks ran out of relevant text; giving it more chunks might narrow the 3.1-point gap.
- **Relative dates.** The fault found by the property suite after the test (Section 5.8) was present in the test run, so the LoCoMo numbers are for the engine before that fix, and the fix is checked only by the 12 scripted date scenarios.

5.11 CONCLUSION AND FUTURE WORK

We built all five modules of the memory engine and measured whether it helps an agent. On LoCoMo, with the same reader under every condition, the engine raised accuracy from 43.0% for an agent that keeps only its recent history, and 80.0% for plain retrieval over the raw text, to 83.1%. That is 6.0 points below giving the reader the whole conversation, while using 30% of its tokens and a context that does not grow with the history. Every criterion fixed before the test passed, on all ten conversations and on the eight never used for development; two judges from different model families agree on 96.7% of the verdicts; and every one of the 99,971 source quotes the engine returned matched the conversation exactly. The gain over plain retrieval is small, 3.1 points, and comes mostly from multi-hop questions. The engine's other properties, which LoCoMo does not measure, were confirmed directly: corrections keep their history and `as_of` recall returns it, the engine abstains without evidence, and it never stores a credential.

The results also show what is not yet good enough. Assertions alone answer 66.7%, far below the source text they came from, so the extractor still misses facts even after Stage C; the graph and consolidation had no measurable effect on LoCoMo; and relative dates were wrong in the test run until the fix found by the property suite. Building memory costs about a minute of GPU time per session on a laptop. Future work, by module:

- M1: API access control and verified backup and restore.
- M2: better extraction coverage, so that assertions can replace more of the source text in the budget, and faster extraction.
- M3: evaluate ambiguous updates, aliases and longer correction histories on a benchmark built for them.

- M4: run LongMemEval and an agent-coding benchmark, and test whether the graph helps on histories with more people and relations than LoCoMo has.
- M5: more coverage-focused training, a date-resolution test set, and independent human review of the judges.

REFERENCES

[1] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguist.*, vol. 12, pp. 157–173, 2024. [Online]. Available: https://aclanthology.org/2024.tacl-1.9/

[2] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking chat assistants on long-term interactive memory," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025, pp. 86 809–86 836, also arXiv:2410.10813. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html

[3] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," arXiv preprint arXiv:2310.08560, Oct. 2023, (rev. Feb. 2024). [Online]. Available: https://arxiv.org/abs/2310.08560

[4] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A temporal knowledge graph architecture for agent memory," arXiv preprint arXiv:2501.13956 [cs.CL], Jan. 2025. [Online]. Available: https://arxiv.org/abs/2501.13956

[5] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," in *Proc. 28th Eur. Conf. Artif. Intell. (ECAI)*, ser. Frontiers in Artificial Intelligence and Applications, vol. 413. IOS Press, Oct. 2025, pp. 2993–3000, also arXiv:2504.19413 [cs.CL].

[6] S. Daga, S. Sreedhar, and D. Shah, "LongMemEval," supermemory research (vendor-hosted evaluation report on LongMemEval-S, not peer reviewed), May 2026, (accessed Oct. 6, 2026). [Online]. Available: https://supermemory.ai/research/longmembench/

[7] C. Latimer, N. Boschi, A. Neeser, C. Bartholomew, G. Srivastava, X. Wang, and N. Ramakrishnan, "Hindsight is 20/20: Building agent memory that retains, recalls, and reflects," arXiv preprint arXiv:2512.12818 [cs.CL], Dec. 2025. [Online]. Available: https://arxiv.org/abs/2512.12818

[8] Neo4j, Inc., "Query the database," in *Neo4j HTTP API Manual, Version 5* (page marked deprecated in 5.26). Neo4j, Inc., 2026, (accessed Oct. 6, 2026). [Online]. Available: https://neo4j.com/docs/http-api/5/query/

[9] M. Zechner *et al.*, "pi: AI agent toolkit (unified LLM API, agent loop, TUI, coding agent CLI)," package @earendil-works/pi-coding-agent, v1.0.2 (version locked by the project's package-lock.json; latest is 1.0.4 on Oct. 6, 2026), GitHub repository, 2026, (accessed Oct. 6, 2026). [Online]. Available: https://github.com/earendil-works/pi

[10] PostgreSQL Global Development Group, "PostgreSQL 17 documentation, chapter 12: Full text search (sections 12.2.2, 12.3.2, 12.3.3, 12.9) and section 64.4: GIN indexes," PostgreSQL 17.11 documentation, 2026, (accessed Oct. 6, 2026). [Online]. Available: https://www.postgresql.org/docs/17/textsearch.html

[11] A. Kane *et al.*, "pgvector: Open-source vector similarity search for Postgres," source code repository and README (0.8.6 installed in the test database, v0.8.7 tag also released), Oct. 2026, (accessed Oct. 6, 2026). [Online]. Available: https://github.com/pgvector/pgvector

[12] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval (SIGIR '09)*, Boston, MA, USA, Jul. 2009, pp. 758–759.

[13] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang, and J. Zhou, "Qwen3 Embedding: Advancing text embedding and reranking through foundation models," arXiv preprint arXiv:2506.05176 [cs.CL], Jun. 2025. [Online]. Available: https://arxiv.org/abs/2506.05176

[14] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023, pp. 10 088–10 115, also arXiv:2305.14314.

[15] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating very long-term conversational memory of LLM agents," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, Aug. 2024, pp. 13 851–13 870, (preprint: arXiv:2402.17753). [Online]. Available: https://aclanthology.org/2024.acl-long.747/

[16] Wikimedia Foundation, "Wikibase/DataModel," MediaWiki.org, 2026, (accessed Oct. 20, 2026). [Online]. Available: https://www.mediawiki.org/wiki/Wikibase/DataModel

[17] T. Lebo, S. Sahoo, and D. McGuinness, "PROV-O: The PROV ontology," W3C Recommendation, Apr. 2013. [Online]. Available: https://www.w3.org/TR/prov-o/

[18] T. Mitchell, W. Cohen, E. Hruschka, P. Talukdar, B. Yang, J. Betteridge, A. Carlson, B. Dalvi, M. Gardner, B. Kisiel, J. Krishnamurthy, N. Lao, K. Mazaitis, T. Mohamed, N. Nakashole, E. Platanios, A. Ritter, M. Samadi, B. Settles, R. Wang, D. Wijaya, A. Gupta, X. Chen, A. Saparov, M. Greaves, and J. Welling, "Never-ending learning," *Commun. ACM*, vol. 61, no. 5, pp. 103–115, May 2018.

[19] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. 36th Annu. ACM Symp. User Interface Softw. Technol. (UIST '23)*, 2023, also arXiv:2304.03442.

[20] TypeSafe AI, "SystemOne API: the Jev decision model," https://api.typesafe.ai/v1/systemone, 2026, model alias jev-latest. Accessed: Oct. 21, 2026.

[21] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

[22] J. Cohen, "A coefficient of agreement for nominal scales," *Educ. Psychol. Meas.*, vol. 20, no. 1, pp. 37–46, 1960.

[23] LangChain, "LangMem SDK: Long-term memory for agents," https://github.com/langchain-ai/langmem, 2025, accessed: Oct. 20, 2026.

[24] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-MEM: Agentic memory for LLM agents," in *Proc. 39th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2025, arXiv:2502.12110 [cs.CL].